

TLS-handskakning + ECDHE

Introduktion

När du besöker en webbplats med HTTPS sker en process kallad TLS

1.3-handskakning osynligt i bakgrunden innan någon data utväxlas. Denna handskakning löser två grundläggande problem samtidigt:

- att bevisa att servern verkligen är den den utger sig för att vara,
- och att etablera en delad hemlighet som krypterar all efterföljande trafik utan att den hemligheten någonsin skickas över internet.

Grunden för denna process är Diffie-Hellman-nyckelutbytet och bygger sin säkerheten på det 'diskreta logaritmproblemet'. Alltså att det är enkelt att beräkna fram ett resultat men är praktiskt taget omöjligt att räkna baklänges för att hitta det ursprungliga värdet. Metoden är matematiskt solid men kräver alltmer stora nycklar för att ligga steget före angripare med allt starkare datorkraft, vilket gör den beräkningsmässigt kostsam i stor skala.

TLS 1.3, standardiserat sedan år 2018, ersatte klassisk DH med Elliptic Curve Diffie-Hellman Ephemeral (ECDHE). Genom att utföra samma logiska nyckelutbyte på punkter längs en tvådimensionell matematisk kurva istället för på en linje levererar ECDHE likvärdig säkerhet med dramatiskt mindre nycklar storleksmässigt, snabbare beräkningar och inbyggd 'forward secrecy by design'. Det innebär att handskakningen kan skalas upp för mindre datorkraft och att en angripare inte kan dekryptera sessionen.

Fas 1: TCP 3-way handshake och ClientHello

1.1

Klienten söker TCP-anslutning med bankens server genom en 3-way handshake i klartext.

1.2

När klienten får ACK-paketet av bankens server skickar klienten sin 'ClientHello' till bankens server för att inleda en TLS-handshake. Den innehåller information om sin TLS-version, krypteringssvit och en 32-byte 'client random' vilket är en unik och tillfällig publik session värde i klartext som används som en del av saltet längre ner. Moderna webbläsare kan kryptera ClientHello med ECH, bankens publika nyckel hämtad via DNS, för att dölja domännamnet för avlyssnare. Syftet är att etablera en säker anslutning mellan klientens webbläsare och bankens server, samt att förebygga replay och man-in-the-middle-attacker.

Fas 2: ServerHello, Autentisering och ECDHE-tal

2.2

Bankens server svarar klienten med en motsvarande hälsning som heter 'ServerHello' och innehåller information om vald krypteringssvit, sitt certifikat, en

färsk 32-byte 'server random' och ett publikt ECDHE-tal signerat med sin privata nyckel. Certifikatet innehåller bankens domännamn, public key och en digital signatur från en CA. ECDHE-talet är resultatet av b gånger G : där b är en engångs, privat tal (ephemeral) och G är en baspunkt med (x, y) -koordinaterna på en graf. I uträkningen i fas 3 utgör ECDHE-talet B .

2.3

I nästa steg verifierar klienten den mottagna informationen lokalt genom att identifiera och verifiera CA och dess signatur. Klienten kör certifikatet genom exempelvis SHA256 och får resultat X . Därefter dekrypteras CA signatur med deras publika nyckel och får resultatet Y . Om resultaten är detsamma tolkar webbläsaren detta som att CA signatur är autentisk och därmed är bankens publika nyckel också autentisk. Till slut kan webbläsaren verifiera bankens signatur av sitt publika ECDHE-tal på samma sätt som med CA.

Fas 3: ECDHE Key Exchange

3.1

Nedan följer en enkel illustration över hur den asymmetriska aspekten går tillväga. Principiellt utgår respektive part ifrån några gemensamma parametrar som är kända för allmänheten och som gäller för alla. Respektive part väljer ett privat tal (a eller b i exemplet nedan) som också kallas för 'ephemeral' på engelska vilket i detta sammanhang betyder att den är kortlivad och raderas så den delade hemligheten har räknats ut. Den har en avgörande roll i att försäkra oss 'Forward Secrecy' vilket betyder att en angripare inte kan återskapa sessionen.

Permanent publika parametrar som är förinstallerade i webbläsaren:

p = stort primtal

G = fast startpunkt på kurvan (x, y -koordinater)

n = övre gräns för privata värden

Till skillnad från klassisk DH tillämpas mod p efter varje enskilt beräkningssteg, vilket håller alla koordinatvärden inom 256 bitar genomgående och gör beräkningarna betydligt effektivare.

Webbläsaren genererar och beräknar lokalt:

a = slumpmässigt heltal mellan 1 och $n-1$. Detta är privat och skickas aldrig ut.

$A = G \times a \pmod{p}$. Detta skickas till servern och blir publik.

Servern genererar och beräknar lokalt:

b = slumpmässigt heltal mellan 1 och $n-1$. Detta är privat och skickas aldrig ut.

$B = G \times b \pmod{p}$.

Eftersom servern redan har skickat sitt B till klientens webbläsare i ServerHello behöver klientens webbläsare skicka över sitt A till servern för att gå vidare. A och B

är 33 bytes stora var.

Webbläsaren beräknar lokalt:

$$S = B \times a \pmod{p}$$

Servern beräknar lokalt:

$$S = A \times b \pmod{p}$$

Båda kommer fram till: **S = samma punkt på kurvan (x,y)**

Delad hemlighet: **Sx = x-koordinaten för S**

Sx matas in i KDF tillsammans med klient- och server random , se fas 4.

Fas 4: Skapandet av symmetriska nycklar

4.1

Respektive part matar x-koordinaten från resultatet S tillsammans med 'salten' dvs de tidigare nämnda 'client random' och 'server random' in i en och samma enkelriktade nyckelhärledningsfunktion HKDF (HMAC-based Extract-and-Expand Key Derivation Function):

$$\text{Salt} = \text{CR} + \text{SR}$$

$$\text{HMAC-SHA256}(\text{salt}, \text{Sx}) = \text{PRK}$$

4.2

Denna PRK används sedan för att skapa två nycklar genom att köras genom HMAC-SHA256 två gånger med olika etiketter. Respektive maskiner har därmed var sin kopia av båda nyckelparen utan att ha utbytt dem. En nyckel används för trafik från klienten och den andra för trafik från bankens server vilket utgör en symmetrisk relation i respektive riktning:

$$\text{HMAC-SHA256}(\text{PRK}, \text{'client key'}) = \text{'client_write_key'}$$

$$\text{HMAC-SHA256}(\text{PRK}, \text{'server key'}) = \text{'server_write_key'}$$

Fas 5: Kryptering och dekryptering av trafik

5.1

Trafiken krypteras och dekrypteras av protokollet AES-256-GCM med hjälp av de symmetriska nycklarna.

Slutsats

TLS 1.3 kombinerar asymmetrisk kryptografi för identitetsverifiering och nyckelutbyte med symmetrisk AES-kryptering för effektiv dataöverföring. Inbyggd 'forward secrecy' innebär att temporära privata värden (a och b i vårt exempel) raderas omedelbart efter varje session, vilket gör det omöjligt för en angripare att dekryptera tidigare trafik.

Framtiden

Kvantdatorer utgör en fundamental utmaning mot både klassisk DH och ECDHE. En tillräckligt kraftfull kvantdator som kör Shors algoritm skulle kunna lösa de diskreta logaritmproblem som båda systemen bygger på. Med andra ord skulle en angripare kunna lagra kopior på kommunikation i väntan på att en kvantdator knäcker krypteringen dvs 'harvest now, decrypt later'. Som svar fastställde NIST sina första post-kvantkryptografistandarder 2024, däribland Kyber för nyckelutbyte som appen Signal numera förlitar sig på och Dilithium för digitala signaturer.